Loop — Home Assignment

Welcome, and thanks for taking the time to work on this. This assignment is designed to give us a realistic look at how you build software end-to-end: how you think about integrations, design APIs, structure code, and make pragmatic trade-offs. We've kept the rules deliberately loose so you have room to make decisions and show us how you work.

The task

Build a small service that integrates with an **external API** and surfaces interesting insights from it.

You choose the integration — **GitHub, Jira, Linear, Trello, GitLab, Bitbucket, Asana, Notion**, or anything else with a public API and meaningful collaboration data. Pick whatever you can get running quickly and whatever lets you produce the most interesting result.

At minimum, your solution must:

1. **Integrate with at least one external API** and fetch enough data to compute something meaningful. The more signals you pull in, the richer your output can be.
2. **Expose at least one HTTP endpoint that returns an interesting insight** over a given period. For example:
    - **Top contributors** by number of commits on the main branch, PRs merged, or lines changed.
    - **Top reviewers** by reviews submitted, review comments, or approvals.
    - **Top closers** by number of issues / tickets / tasks closed.
    - Anything else you find genuinely interesting — surprise us.
1. **Expose a second endpoint that uses an LLM to synthesize a short narrative over the numbers** — a few sentences explaining what's interesting or unusual about the data, with a root-cause hypothesis where the signal supports one, a confidence score, and an evidence chain pointing back to the underlying numbers.

Document briefly what your metric means and why you chose it.

1. **Be queryable over HTTP**. A small, well-designed REST or GraphQL API is perfect. The reviewer should be able to hit it directly (curl, HTTPie, Postman, etc.).
2. **Be runnable locally** with clear setup instructions. Some setup (env vars, an API token, `docker compose up`, etc.) is fine — just document it.
3. **Follow web service best practices.** We expect production-grade thinking: sensible HTTP semantics eg. Treat this as code you'd ship, not a throwaway script. See **What we look for** below for what we pay attention to.

That's it for the hard requirements.

Optional — but nice to have

Anything below is a bonus, not a requirement. Pick what interests you; ignore the rest.

- **A small frontend** to visualize the insights. React is preferred, but anything reasonable is fine. Even a single page with a couple of tables and a date-range picker goes a long way for a demo.
- **More insights or signals** — issue turnaround time, time-to-merge / time-to-close, stale items, review or assignee load balance across the team, trend lines over time, etc.
- **Caching** of upstream API responses so repeated queries don't burn your rate limit.
- **More than one integration**, or a design that makes adding the next one straightforward.
- **Background sync** instead of fetching on demand — e.g. a worker that periodically pulls data into a local store.
- **Tests** for the parts you consider most important. We don't need 100% coverage; we want to see what you choose to test and why.
- **Eval harness** - ship a small evaluation suite you'd actually run before changing a prompt or swapping a model.
- **Containerization** (`Dockerfile`, `docker-compose.yml`) so the reviewer can run everything with one command.

Feel free to extend the scope in any direction that excites you. If you build something we didn't ask for, briefly mention it in your submission notes so we don't miss it.

Technical constraints

- **Language:** TypeScript is preferred. Kotlin and Python are equally fine. If you have a strong reason to use something else, drop us a note and go ahead — we care more about the result than the stack.
- **Framework / libraries:** your choice. Use what you'd reach for at work.
- **Database / storage:** not strictly required, but preferred — see the performance note below. In-memory works for the core task; reaching for a real store (Postgres, SQLite, Redis, whatever fits) is a good way to show you've thought about caching and repeat queries.
- **API access:** use a personal access token or equivalent with read-only scopes. Public data is fine; don't ask us to provide credentials for private orgs / workspaces.

What we look for

We're not grading on a checklist — we're reading the code the way we'd read a teammate's PR. Concretely, we pay attention to:

- **Correctness.** The endpoints return what they claim to return, edge cases are handled sensibly, and the numbers actually add up against the source data.
- **Code quality.** Clear structure, sensible boundaries, readable naming, no dead code. We'd rather see a small codebase done well than a large one done loosely.
- **Security.** No tokens in the repo or logs, no obvious injection or SSRF paths, sensible handling of untrusted input. Treat this as production code, not a throwaway script.
- **Performance.** Reasonable use of the upstream API, sensible caching where it helps, no obviously quadratic loops over large result sets, no blocking the event loop on big payloads. Persisting fetched data in a database (rather than re-hitting the upstream API on every request) is preferred but not a strong requirement — if you skip it, we'll expect your design to make clear how you'd add it.
- **Operability.** It's easy to run, easy to demo, and clear what the moving parts are. A `README` with a 60-second quickstart is worth a lot.
- **Judgement.** The trade-offs you made and the things you chose not to do — explained briefly in your submission notes — tell us as much as the code does.

On using AI coding assistants

We use AI coding tools at Loop every day and we encourage you to do the same here — Cursor, Claude Code, Copilot, whatever you prefer. The bar isn't "did you write every line yourself"; it's **"is this code you'd be comfortable putting your name on and shipping to production?"**

That means: review what the agent produces, push back on it when it's wrong, and make sure the final result meets the same security, performance, and quality bar as anything else. If anything in the codebase surprises you on a re-read, fix it before submitting.

Submitting

Please submit either:

- a **link to a public Git repository** (GitHub, GitLab, etc.), or
- a **zip / tarball** of the project with the `.git` directory included.

In your submission, please include a short `NOTES.md` (or section in the README) covering:

1. **How to run it locally** — exact commands, env vars, any prerequisites.
2. **A 1–2 paragraph tour** of the architecture and the main decisions you made.
3. **What you'd do next** if you had another day.
4. **Anything you used AI for** and roughly how — we're curious, not judgmental.

A note on demoing

When you come in for the follow-up conversation, plan to spend ~10 minutes walking us through:

- a live demo of the running service (and UI, if you built one),
- a quick code tour of the parts you're most proud of,
- a couple of trade-offs you'd revisit.

We'll have read the code beforehand, so this is a conversation — not a presentation.

Good luck, and have fun. We're looking forward to seeing what you build.